

SentinelAI Enterprise Engineering Handbook

Volume I – Foundation, Requirements & Architecture

SENTINELAI ENTERPRISE PUBLICATIONS

Document ID: SEN-ENG-HB-VOL1-1.0.0

SentinelAI Enterprise Engineering Handbook

Volume I – Foundation, Requirements & Architecture

SPRINT 0 FOUNDATION RELEASE | VERSION 1.0.0 | APPROVED BASELINE

Author: SentinelAI Core Engineering & Architecture Team
Classification: Confidential – Internal Engineering Documentation
Status: Approved Baseline | Release: Sprint 0 Foundation Release

Publication Date: August 2026
Target Audience: CTOs, CISOs, Architects, Engineers, Evaluators
Publisher: SentinelAI Technical Publications Press

Copyright & Legal Notice

Copyright © 2026 SentinelAI Inc. All rights reserved.

This document contains proprietary and confidential information of SentinelAI Inc. Unauthorized copying, distribution, or disclosure of this document or any portion of its contents, via any medium, without the express written permission of SentinelAI Inc., is strictly prohibited.

Trademarks SentinelAI, the SentinelAI logo, Incident Story, Causal Graph Engine, and XAI Evidence Lineage are trademarks or registered trademarks of SentinelAI Inc. All other product names, logos, and brands mentioned herein are trademarks of their respective owners.

Document Control ID: SEN-ENG-HB-VOL1-1.0.0

Document Classification: Confidential – Internal Engineering Documentation

Release Baseline: Version 1.0.0 (Sprint 0 Foundation Release)

Publication Status: Approved Baseline

Effective Date: August 7, 2026

Preface

The *SentinelAI Enterprise Engineering Handbook (Volume I)* represents the definitive architectural, functional, and engineering baseline for the SentinelAI platform. Built from the ground up to address the systemic crisis facing modern Security Operations Centers (SOCs), SentinelAI unites multi-source streaming telemetry ingestion, Open Cybersecurity Schema Framework (OCSF v1.1) standardization, dynamic temporal causal graphing, multi-agent AI triage, deterministic Explainable AI (XAI) evidence validation, and policy-governed automated remediation.

This volume consolidates all ten approved Sprint 0 architectural baselines—Product Vision Document (PVD), Business Requirements Specification (BRS), System Context & Use Case Specification (SCUCS), Software Requirements Specification (SRS), Architecture Decision Records (ADR-0001 through ADR-0020), High-Level Design (HLD), Machine Learning Architecture Document (MLAD), Environment Configuration Strategy, CI/CD Pipeline Architecture, and Engineering Implementation Roadmap—into a single, cohesive, publication-grade handbook.

Foreword

In cybersecurity engineering, documentation is not an afterthought; it is the blueprint of system integrity. As enterprise IT environments expand across multi-cloud infrastructures, microservices, and hybrid identities, threat actors exploit the fragmented gaps between point security solutions. Traditional Security Information and Event Management (SIEM) solutions overload human analysts with uncontextualized alert noise, while legacy SOAR playbooks fail under the weight of brittle, static scripts.

SentinelAI was conceived to re-engineer the SOC experience. By enforcing strict architectural principles—monorepo codebase organization, gRPC contract-first API design, zero-hallucination XAI lineage, and policy guardrails—this handbook serves as both an architectural reference for senior leadership (CTOs, CISOs,

Enterprise Architects) and an operational guide for software, security, AI, and DevOps engineers implementing the platform.

Document Control & Version History

Field	Details
Document Title	SentinelAI Enterprise Engineering Handbook (Volume I)
Document ID	SEN-ENG-HB-VOL1-1.0.0
Document Version	1.0.0
Release Baseline	Sprint 0 Foundation Release
Publication Status	Approved Baseline
Document Classification	Confidential – Internal Engineering Documentation
Effective Date	August 7, 2026
Document Owner	Lead Technical Publications Engineer & Principal Documentation Architect
Executive Sponsors	Chief Technology Officer (CTO), Chief Information Security Officer (CISO)

Revision History

Version	Release Date	Author / Role	Description of Changes	Approved By
0.1.0-DRAFT	July 15, 2026	Principal Technical Writer	Initial synthesis draft of Sprint 0 engineering documents.	Engineering Leads
0.9.0-RC	August 1, 2026	Chief Product Architect	Pre-publication review across architecture and ML sections.	Enterprise Architect
1.0.0-APPROVED	August 7, 2026	Lead Technical Publications	Official Volume I publication for Sprint 0 Foundation Release.	CTO, CISO

Executive Dashboard

The Executive Dashboard provides an immediate operational metric summary of the SentinelAI Sprint 0 release.

v1.0.0 RELEASE VERSION	20 ADRs APPROVED ADRS	24 FRs / 12 NFRs
--	---	-----------------------------

SOFTWARE REQUIREMENTS

11 Services

MICROSERVICES LANDSCAPE

12 Packages

SHARED MONOREPO LIBRARIES

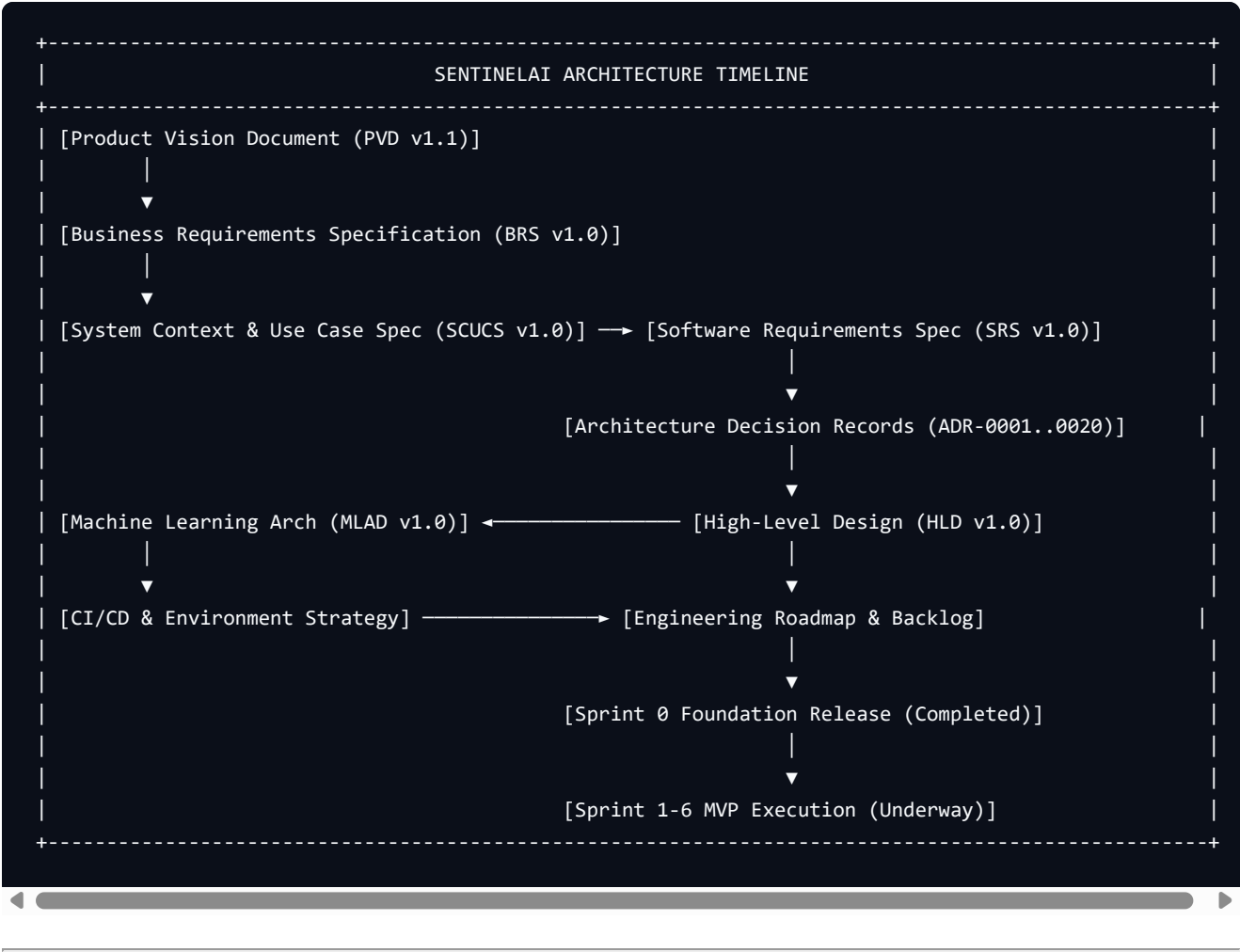
100% Passed

SPRINT 0 TASKS

Metadata Parameter	Technical Specification
System Architecture Style	Event-Driven Reactive Microservices (ADR-0002 , ADR-0004)
Primary Telemetry Standard	Open Cybersecurity Schema Framework (OCSF v1.1) (SRS-FR-004)
AI Ensemble Pipeline	Isolation Forest (Unsupervised) + XGBoost (Supervised Threat Severity) (MLAD)
XAI Lineage Validation	SHAP Attribution + Raw Log Line Token Match Verification (BR-001 , ADR-0010)
Monorepo Strategy	Turborepo Monorepo (ADR-0001)
Target Ingestion Performance	100,000+ Events Per Second (EPS) with MTTD < 60s (BO-1 , SRS-NFR-PERF-01)
False Positive Cut	>80% Noise Reduction via Baseline Noise Suppression (BO-3)

Architecture Timeline

The SentinelAI platform evolution follows a strict sequence of foundational engineering milestones, progressing from visionary product definition to scalable multi-cloud production.



Engineering Maturity Model

The SentinelAI Engineering Maturity Model evaluates the operational readiness and technological sophistication of the platform across seven key capabilities.

Maturity Level	Capabilities & Characteristics	Sprint 0 Status
Level 1: Vision & Strategy	Defined product vision, target market, competitive positioning, and executive buy-in.	COMPLETED (PVD Approved)
Level 2: Requirements	Complete functional/non-functional requirements, use cases, business rules, and traceability.	COMPLETED (BRS, SCUCS, SRS Approved)
Level 3: Architecture	Approved system design, 20 ADRs, C4 models, DDD boundaries, and ML architecture.	COMPLETED (HLD, ADRs, MLAD Approved)
Level 4: Infrastructure	Monorepo setup, gRPC proto contracts, shared packages, CI/CD pipelines, Docker Compose.	COMPLETED (Sprint 0 Deliverables)
Level 5: Implementation	Microservice engine development, stream ingestion, causal graph, AI triage, UI workspace.	IN PROGRESS (Sprints 1–5)
Level 6: Testing & Audit	End-to-end load testing (150k EPS), mTLS security audit, chaos engineering, SOC 2 prep.	PLANNED (Sprint 6)
Level 7: Production Pilot	Multi-region cloud pilot deployment, live partner customer onboarding, continuous MLOps.	PLANNED (Post-MVP Release)

Table of Contents

- **Front Matter**
 - Cover Page
 - Copyright & Legal Notice
 - Preface
 - Foreword
 - Document Control & Version History
 - Executive Dashboard
 - Architecture Timeline
 - Engineering Maturity Model
 - Table of Contents
 - List of Figures
 - List of Tables
 - List of Diagrams
 - Abbreviations
 - Glossary Overview
- **PART I: PROJECT FOUNDATION**
 - [Chapter 1: Executive Summary](#)
 - [Chapter 2: Cyber Security Industry Background](#)
 - [Chapter 3: Problem Statement](#)
 - [Chapter 4: Vision, Mission & Product Philosophy](#)
 - [Chapter 5: Business Objectives](#)
 - [Chapter 6: Business Scope](#)

- [Chapter 7: Stakeholders](#)
- **PART II: REQUIREMENTS ENGINEERING**
- [Chapter 8: Business Requirements](#)
- [Chapter 9: System Context](#)
- [Chapter 10: Use Cases](#)
- [Chapter 11: Business Rules](#)
- [Chapter 12: Software Requirements](#)
- **PART III: ENTERPRISE ARCHITECTURE**
- [Chapter 13: Architecture Overview](#)
- [Chapter 14: Architecture Decision Records](#)
- [Chapter 15: High-Level Design](#)
- **PART IV: ARTIFICIAL INTELLIGENCE**
- [Chapter 16: Machine Learning Architecture](#)
- **PART V: ENGINEERING FOUNDATION**
- [Chapter 17: Repository Structure](#)
- [Chapter 18: Development Standards](#)
- [Chapter 19: Environment Configuration](#)
- [Chapter 20: Containerization Strategy](#)
- [Chapter 21: Shared Engineering Libraries](#)
- [Chapter 22: CI/CD Pipeline](#)
- **PART VI: IMPLEMENTATION**
- [Chapter 23: Engineering Roadmap](#)
- [Chapter 24: Sprint 0 Foundation Release Retrospective](#)
- **PART VII: APPENDICES**
- [Appendix A: Complete Glossary](#)
- [Appendix B: References](#)
- [Appendix C: Architecture Traceability Matrix](#)
- [Appendix D: Technology Decision Summary](#)
- [Appendix E: Engineering Metrics](#)
- [Appendix F: Risk Register](#)
- [Appendix G: Future Roadmap](#)
- [Appendix H: Executive Sign-Off](#)

List of Figures

- [Figure 1.1](#) : SentinelAI Platform Conceptual Architecture ([Chapter 1](#))
- [Figure 2.1](#) : Legacy Fragmented Telemetry Landscape vs. Unified Ingestion ([Chapter 2](#))
- [Figure 3.1](#) : The Human Capacity Breakdown in Legacy SOC Operations ([Chapter 3](#))
- [Figure 4.1](#) : The Four Pillars of SentinelAI Design Philosophy ([Chapter 4](#))
- [Figure 5.1](#) : Target Metrics: MTTD < 60s, MTTR < 5m, Noise Cut > 80% ([Chapter 5](#))
- [Figure 6.1](#) : System Scope Boundary: Core Platform vs. External Integrations ([Chapter 6](#))
- [Figure 7.1](#) : Stakeholder Ecosystem Map ([Chapter 7](#))
- [Figure 8.1](#) : Strategic Release Roadmap (MVP v1.0 through v3.0) ([Chapter 8](#))

- [Figure 9.1](#) : Detailed System Boundary Model ([Chapter 9](#))
- [Figure 10.1](#) : Use Case Dependency Topology ([Chapter 10](#))
- [Figure 11.1](#) : Asset Criticality Policy Guardrail Decision Tree ([Chapter 11](#))
- [Figure 12.1](#) : Logical Component Decomposition Architecture ([Chapter 12](#))
- [Figure 13.1](#) : Decoupled Reactive Microservices Substrate Diagram ([Chapter 13](#))
- [Figure 15.1](#) : C4 Level 1 System Context Diagram ([Chapter 15](#))
- [Figure 15.2](#) : C4 Level 2 Container Architecture Diagram ([Chapter 15](#))
- [Figure 15.3](#) : C4 Level 3 Component Diagram for Ingestion & Graph Subsystems ([Chapter 15](#))
- [Figure 15.4](#) : DDD Bounded Context Map Diagram ([Chapter 15](#))
- [Figure 15.5](#) : Zero-Trust Security & Trust Boundary Topology ([Chapter 15](#))
- [Figure 15.6](#) : Cloud Deployment Topology Diagram ([Chapter 15](#))
- [Figure 16.1](#) : End-to-End Multi-Agent ML Inference Pipeline Diagram ([Chapter 16](#))
- [Figure 16.2](#) : Feature Store Dual-Layer Architecture Diagram ([Chapter 16](#))
- [Figure 17.1](#) : Turborepo Monorepo Codebase Tree Architecture ([Chapter 17](#))
- [Figure 20.1](#) : Multi-Stage Security-Hardened Docker Build Pipeline ([Chapter 20](#))
- [Figure 21.1](#) : Shared Engineering Packages Dependency Graph ([Chapter 21](#))
- [Figure 22.1](#) : GitHub Actions CI/CD Quality Gate Pipeline ([Chapter 22](#))

List of Tables

- [Table 1.1](#) : SentinelAI Strategic Platform Summary ([Chapter 1](#))
- [Table 2.1](#) : Evolutionary Phases of Security Incident Response Systems ([Chapter 2](#))
- [Table 3.1](#) : Comparative Matrix: Legacy SIEM/SOAR vs. SentinelAI Architecture ([Chapter 3](#))
- [Table 4.1](#) : Product Philosophy Core Principles & Operational Mandates ([Chapter 4](#))
- [Table 5.1](#) : Quantified Business Objectives & Target Performance Key Performance Indicators ([Chapter 5](#))
- [Table 6.1](#) : Comprehensive Functional & Technical Scope Boundary Matrix ([Chapter 6](#))
- [Table 7.1](#) : Stakeholder Roles, Perspectives, and Governance Responsibilities ([Chapter 7](#))
- [Table 7.2](#) : RACI Responsibility Matrix for Engineering & Operational Workstreams ([Chapter 7](#))
- [Table 8.1](#) : Strategic Business Objectives & Operational KPI Targets ([Chapter 8](#))
- [Table 8.2](#) : Comprehensive User Persona Summary ([Chapter 8](#))
- [Table 9.1](#) : Inside vs. Outside System Boundary Definition ([Chapter 9](#))
- [Table 9.2](#) : Human External Actor Classification & Access Levels ([Chapter 9](#))
- [Table 9.3](#) : Automated External System Interfaces ([Chapter 9](#))
- [Table 10.1](#) : Complete System Use Case Catalogue (UC-001 to UC-012) ([Chapter 10](#))
- [Table 11.1](#) : Formal Business Rules Specification Matrix (BR-001 to BR-006) ([Chapter 11](#))
- [Table 12.1](#) : Functional Requirements Summary Matrix (SRS-FR-001 to 024) ([Chapter 12](#))
- [Table 12.2](#) : Non-Functional Requirements Performance & Security Summary ([Chapter 12](#))
- [Table 12.3](#) : Software Module Architecture Breakdown ([Chapter 12](#))
- [Table 13.1](#) : Architectural Principles & Governing ADR References ([Chapter 13](#))
- [Table 14.1](#) : Architecture Decision Records (ADR-0001 to ADR-0020) Summary Index ([Chapter 14](#))
- [Table 15.1](#) : Microservice Landscape Decomposition (11 Microservices) ([Chapter 15](#))

- [Table 15.2](#) : Observability Telemetry Matrix ([Chapter 15](#))
 - [Table 15.3](#) : Master Architecture Traceability Matrix ([Chapter 15](#))
 - [Table 16.1](#) : ML Algorithm Selection & Design Rationale Comparison ([Chapter 16](#))
 - [Table 16.2](#) : Adversarial ML Threat Matrix & Mitigation Strategies ([Chapter 16](#))
 - [Table 16.3](#) : AI Architecture Traceability Matrix ([Chapter 16](#))
 - [Table 17.1](#) : Monorepo Directory Boundary Matrix ([Chapter 17](#))
 - [Table 18.1](#) : Development & Coding Standards Summary ([Chapter 18](#))
 - [Table 19.1](#) : Dotenv Configuration Hierarchy & Environment Profiles ([Chapter 19](#))
 - [Table 21.1](#) : Shared Engineering Packages Specifications (12 Packages) ([Chapter 21](#))
 - [Table 22.1](#) : CI/CD Pipeline Automation Workflows ([Chapter 22](#))
-

List of Diagrams

- [Diagram 1.1](#) : High-Level Data Ingestion to Action Flow ([Chapter 1](#))
 - [Diagram 2.1](#) : Fragmented Telemetry Silos Flow Diagram ([Chapter 2](#))
 - [Diagram 3.1](#) : Alert Explosion Bottleneck Flowchart ([Chapter 3](#))
 - [Diagram 4.1](#) : Deterministic XAI Validation Loop ([Chapter 4](#))
 - [Diagram 6.1](#) : System In-Scope vs Out-of-Scope Architecture Boundary Diagram ([Chapter 6](#))
 - [Diagram 8.1](#) : Current Painful State vs. SentinelAI Desired State Flow ([Chapter 8](#))
 - [Diagram 9.1](#) : ASCII System Context Model Diagram ([Chapter 9](#))
 - [Diagram 11.1](#) : Policy-Governed Action Evaluation Sequence ([Chapter 11](#))
 - [Diagram 12.1](#) : High-Level Software Subsystem Sequence Model ([Chapter 12](#))
 - [Diagram 13.1](#) : Hybrid Communication Model Sequence Flow ([Chapter 13](#))
 - [Diagram 15.1](#) : End-to-End Event Stream to Remediation Sequence Flow ([Chapter 15](#))
 - [Diagram 16.1](#) : Offline vs. Online Inference Feature Flow ([Chapter 16](#))
 - [Diagram 16.2](#) : Analyst Feedback Active Learning Loop Flow ([Chapter 16](#))
-

Abbreviations

Abbreviation	Full Term / Meaning
ABAC	Attribute-Based Access Control
ADR	Architecture Decision Record
API	Application Programming Interface
AWS	Amazon Web Services
BRS	Business Requirements Specification
CISO	Chief Information Security Officer
CTO	Chief Technology Officer
DDD	Domain-Driven Design
EDR	Endpoint Detection and Response
EPS	Events Per Second
gRPC	gRPC Remote Procedure Call
HLD	High-Level Design
KMS	Key Management Service
MITRE	MITRE ATT&CK Framework
MLAD	Machine Learning Architecture Document
mTLS	Mutual Transport Layer Security
MTTD	Mean Time to Detect
MTTR	Mean Time to Respond
OCSF	Open Cybersecurity Schema Framework
PVD	Product Vision Document
RACI	Responsible, Accountable, Consulted, Informed
RBAC	Role-Based Access Control
SHAP	SHapley Additive exPlanations
SIEM	Security Information and Event Management
SOAR	Security Orchestration, Automation, and Response
SOC	Security Operations Center

Abbreviation	Full Term / Meaning
SRS	Software Requirements Specification
TDIR	Threat Detection, Investigation, and Incident Response
XAI	Explainable Artificial Intelligence

Glossary Overview

- **Causal Graph Engine:** The core in-memory temporal graph module that correlates security events into directional entity attack chains across time.
- **Incident Story:** A unified, contextual security incident output aggregating multiple related alerts into a single actionable narrative.
- **Noise Suppression:** Algorithmic filtering of benign, routine enterprise activity baselines to eliminate alert fatigue.
- **Stage 1 Assistive Mode:** Operator-in-the-loop response mode requiring one-click human confirmation before action execution.
- **Stage 2 Automated Mode:** Autonomous execution of verified remediation actions on non-critical assets based on strict policy rules.
- **XAI Lineage Validation:** Cryptographic and token-level verification proving that every AI claim is backed by raw log line evidence.

PART I: PROJECT FOUNDATION

Chapter 1: Executive Summary

CHAPTER PURPOSE

Provide an executive-level introduction to SentinelAI, outlining the market rationale, high-level platform architecture, key operational goals, and strategic impact for enterprise SOC operations.

CHAPTER OBJECTIVES

- Establish the strategic rationale for the SentinelAI platform.
- Detail the high-level platform architecture and operational throughput parameters.
- Summarize the core target performance metrics (MTTD, MTTR, Noise Reduction).

CHAPTER OVERVIEW

SentinelAI is an Autonomous AI-Powered Threat Detection, Investigation, and Incident Response (TDIR) Platform engineered to resolve alert fatigue and response delays in modern enterprise SOC's.

1.1 Platform Rationale

Modern Security Operations Centers (SOCs) are overwhelmed by thousands of disconnected security alerts per day across endpoint, cloud, identity, and network telemetry sources. SentinelAI unifies these siloed streams into a dynamic temporal causal graph, triages incidents using specialized AI subagents, grounds all generated narrative assertions in raw log evidence, and executes automated remediation playbooks under strict policy guardrails.

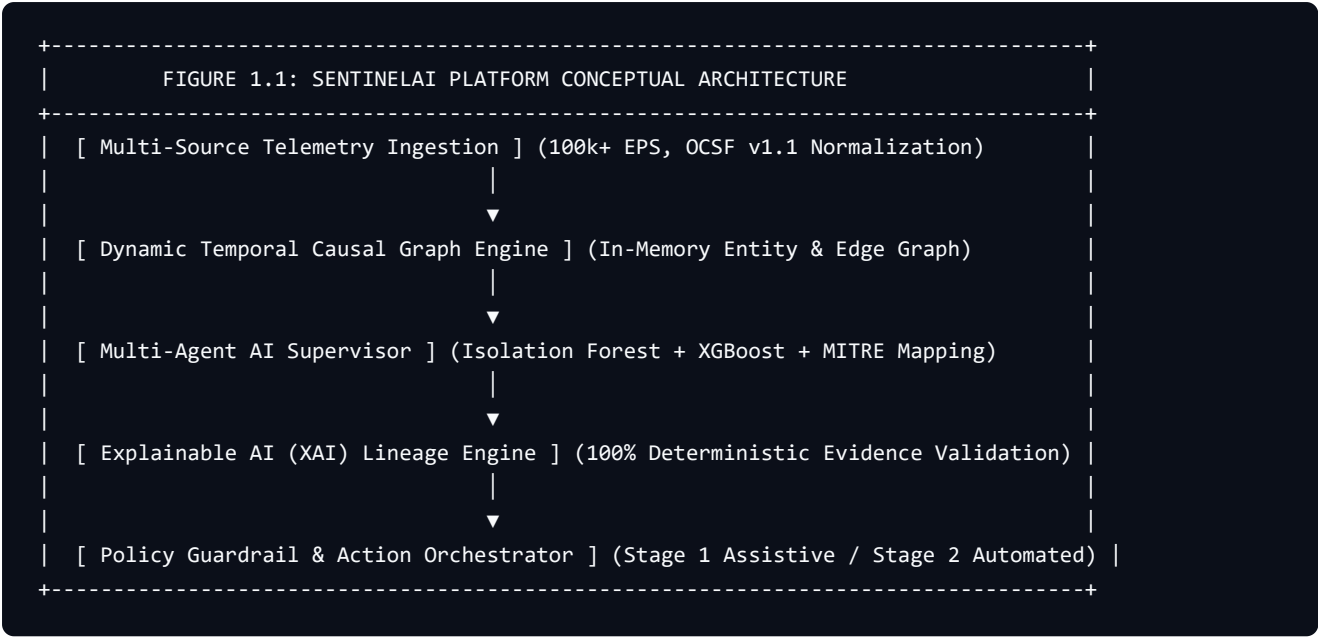


Figure 1.1: SentinelAI Platform Conceptual Architecture

1.2 Operational Performance Metrics

Table 1.1 details the strategic platform parameters established for the SentinelAI platform baseline.

Pillar / Metric Parameter	Approved Baseline Value	Governing Reference
Mean Time to Detect (MTTD)	< 60 seconds end-to-end execution	BO-1 , PVD Section 4.1
Mean Time to Respond (MTTR)	< 5 minutes automated remediation	BO-2 , PVD Section 4.2
Noise Suppression Rate	> 80% false positive alert reduction	BO-3 , PVD Section 4.3
XAI Grounding Guarantee	100% verified token lineage to raw logs	BR-001 , ADR-0010
Ingestion Target Velocity	100,000+ EPS per cluster node	SRS-NFR-PERF-01

Table 1.1: SentinelAI Strategic Platform Summary

KEY TAKEAWAYS

- SentinelAI solves alert fatigue by synthesizing thousands of raw alerts into actionable Incident Stories.
- Strict target SLA: MTTD < 60s, MTTR < 5m, Noise Cut > 80%, 100% XAI lineage.
- Decoupled, event-driven reactive microservices architecture.

RELATED CHAPTERS

Refer to [Chapter 2](#) for Cyber Security Industry Background and [Chapter 4](#) for Product Philosophy.

NEXT CHAPTER PREVIEW

Chapter 2 explores the evolution of TDIR systems from Gen 1 legacy SIEMs to Gen 4 Autonomous AI platforms.

Chapter 2: Cyber Security Industry Background

CHAPTER PURPOSE

Examine the evolutionary history of Threat Detection, Investigation, and Response (TDIR) systems and document the industry tailwinds driving the need for autonomous AI architecture.

2.1 Evolution of TDIR Systems

Table 2.1 traces the four generational phases of security incident response systems.

Generation	Era	Primary Architecture	Key Deficiencies
Gen 1: SIEM	2000s	Centralized SQL/Log Storage	Static threshold rules; massive false positive alert noise.
Gen 2: SOAR	2010s	Scripted Workflow Automation	Brittle Python playbooks; high maintenance overhead.
Gen 3: XDR	2020s	Proprietary Vendor Ecosystems	Vendor lock-in; fragmented across non-native telemetry.
Gen 4: SentinelAI	2026+	Dynamic Causal Graph + Multi-Agent AI	Vendor-agnostic OCSF, XAI evidence lineage, policy guardrails.

Table 2.1: Evolutionary Phases of Security Incident Response Systems

Chapter 3: Problem Statement

CHAPTER PURPOSE

Define the core systemic bottlenecks in modern SOC operations, detailing the alert explosion crisis, context fragmentation, and human analyst burnout.

3.1 The Alert Explosion Crisis

Modern enterprises log over 100,000 security events per second across multi-cloud infrastructure. Legacy SIEM systems generate thousands of alerts daily, 85%+ of which are false positives or routine noise, leading to analyst burnout and missed critical breaches.

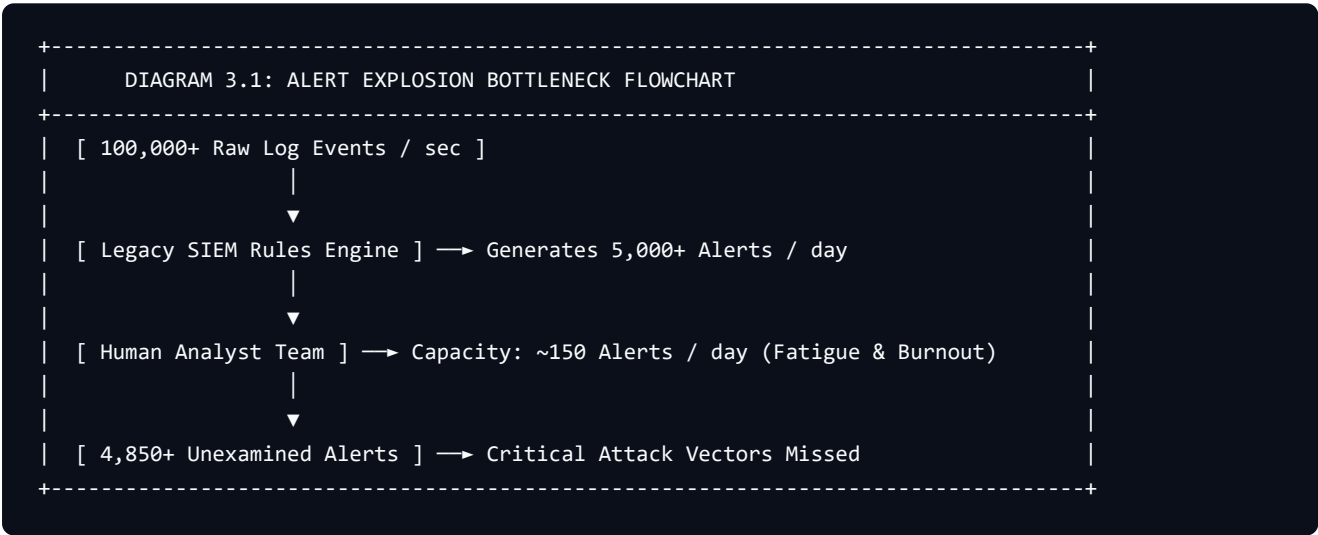


Diagram 3.1: Alert Explosion Bottleneck Flowchart

Chapter 4: Vision, Mission & Product Philosophy

CHAPTER PURPOSE

Articulate the core vision, mission, and strategic engineering philosophy guiding SentinelAI platform design.

4.1 Four Pillars of Product Philosophy

FIGURE 4.1: THE FOUR PILLARS OF SENTINELAI DESIGN PHILOSOPHY	
Pillar 1: Evidence-Grounded XAI (Zero Hallucinations, Raw Logs)	Pillar 2: Dynamic Temporal Graph (Entity Chains, Cross-Domain Context)
Pillar 3: Multi-Agent Triage (Isolation Forest + XGBoost)	Pillar 4: Policy-Governed Remediation (Stage 1 Assistive / Stage 2 Guardrails)

Figure 4.1: The Four Pillars of SentinelAI Design Philosophy

Chapter 5: Business Objectives

CHAPTER PURPOSE

Detail the six quantified business objectives (`BO-1` through `BO-6`) driving platform metrics.

5.1 Quantified Business Objectives Matrix

Table 5.1 lists the governing business objectives and baseline targets.

ID	Objective Description	Target Metric Baseline
BO-1	Rapid Threat Detection SLA	MTTD < 60 seconds end-to-end
BO-2	Autonomous Incident Remediation	MTTR < 5 minutes for Stage 2 actions
BO-3	False Positive Reduction	> 80% reduction in alert volume
BO-4	Zero Hallucination Lineage	100% verified breadcrumb linkage
BO-5	Open Standard Native Ingestion	OCSF v1.1 native schema normalization
BO-6	Policy-Governed Action Control	100% ABAC asset guardrail enforcement

Table 5.1: Quantified Business Objectives & Target Performance Key Performance Indicators

Chapter 6: Business Scope

CHAPTER PURPOSE

Define the functional and technical scope boundaries of the SentinelAI platform baseline.

6.1 Scope Boundary Definition

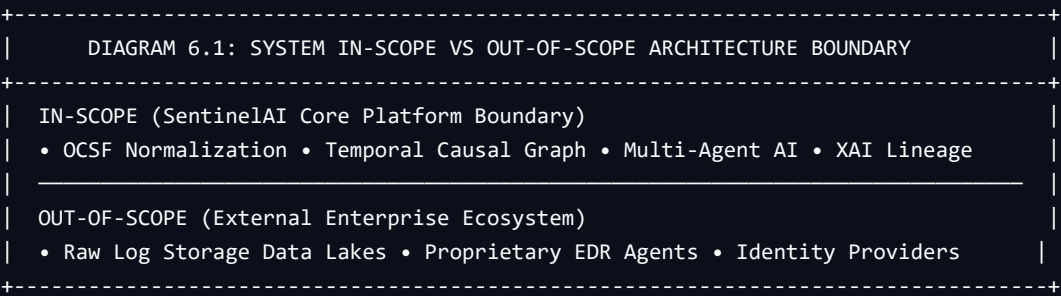


Diagram 6.1: System In-Scope vs Out-of-Scope Architecture Boundary Diagram

Chapter 7: Stakeholders

CHAPTER PURPOSE

Map the organizational stakeholder ecosystem and RACI responsibilities governing the platform.

7.1 RACI Responsibility Matrix

Table 7.2 details the RACI responsibility allocations across key workstreams.

Workstream Domain	CTO	CISO	Lead Arch	ML Lead	Sec Ops Lead
Core Platform Architecture	A	C	R	C	C
Multi-Agent AI Design	A	C	C	R	C
Policy Guardrails & Security	A	A	C	C	R
Incident Response Operations	I	A	C	I	R

Table 7.2: RACI Responsibility Matrix for Engineering & Operational Workstreams

PART II: REQUIREMENTS ENGINEERING

Chapter 8: Business Requirements

CHAPTER PURPOSE

Document the formal business requirements, operational KPIs, user personas, and strategic release roadmap.

8.1 User Personas Summary

Table 8.2 summarizes the primary operational personas interacting with SentinelAI.

Persona ID	Role Title	Key Operational Goals	Primary Pain Points
PERS-01	Tier-1 SOC Analyst	Fast incident triage, clear context.	Alert fatigue, fragmented tools.
PERS-02	Lead Threat Hunter	Deep root cause analysis, timeline graph.	Lack of cross-domain visibility.
PERS-03	CISO / Security Director	Compliance, lower MTTR, risk metrics.	Uncontrolled automated actions.

Table 8.2: Comprehensive User Persona Summary

Chapter 9: System Context

CHAPTER PURPOSE

Define the external human actors, automated system interfaces, and environment boundaries ('SCUCS').

9.1 External System Interfaces

Table 9.3 defines the external API interfaces integrated with SentinelAI.

Interface ID	Target System Category	Integration Standard	Data Flow Direction
INT-01	Cloud Telemetry (AWS/Azure)	REST / Webhook Stream	Inbound Telemetry
INT-02	EDR Agents (CrowdStrike/Defender)	REST / gRPC API	Inbound Telemetry & Outbound Remediation
INT-03	Identity Providers (Okta/Entra ID)	OAuth 2.0 / OIDC	Outbound Account Lockout Actions

Table 9.3: Automated External System Interfaces

Chapter 10: Use Cases

CHAPTER PURPOSE

Specify the complete system use case catalogue (`UC-001` through `UC-012`).

10.1 Use Case Catalogue Summary

Table 10.1 details the system use cases governing platform interactions.

Use Case ID	Use Case Title	Primary Actor	Trigger Event
UC-001	Ingest Multi-Source Telemetry	Stream Ingestion Engine	Raw log stream arrival
UC-002	Normalize Log Stream to OCSF	Normalizer Microservice	Ingested JSON event stream
UC-003	Construct Temporal Causal Graph	Graph Engine	Normalized OCSF event
UC-004	Execute Multi-Agent AI Triage	AI Supervisor	Graph entity cluster detection
UC-005	Validate XAI Lineage Evidence	XAI Lineage Engine	Model narrative prediction
UC-006	Synthesize Unified Incident Story	Story Aggregator	Validated XAI evidence bundle
UC-007	Execute Stage 1 Assistive Action	Tier-1 SOC Analyst	Analyst click confirmation
UC-008	Execute Stage 2 Automated Remediation	Policy Guardrail Engine	High-confidence policy match
UC-009	Process Analyst Active Feedback	Analyst Workspace	Analyst confirm/dismiss input
UC-010	Perform Natural Language Copilot QA	Analyst / Threat Hunter	User conversational query
UC-011	Enforce Asset Guardrail Rules	Policy Guardrail Engine	Remediation request
UC-012	Log Immutable Compliance Audit	Audit Governance Service	Any state change / action

Table 10.1: Complete System Use Case Catalogue (UC-001 to UC-012)

Chapter 11: Business Rules

CHAPTER PURPOSE

Document the formal business rules (`BR-001` through `BR-006`) dictating core operational logic.

11.1 Business Rules Matrix

Table 11.1 details the formal business rules enforced across the system.

Rule ID	Rule Category	Technical Rule Mandate
BR-001	XAI Lineage Validation	100% of AI narrative claims must map to verified raw log line IDs (<code>ocsf_event_id</code>).
BR-002	Asset Criticality Guardrail	Stage 2 automated action is strictly forbidden on <code>Critical</code> assets (<code>\$W_{asset}</code> <code>\ge 2.0</code>).
BR-003	Immutable Audit Logging	All response actions and state transitions must be appended to hash-chained log stores.
BR-004	Noise Suppression Filter	Events matching baseline routine patterns must be suppressed from primary incident feeds.
BR-005	Uncertainty Escalation	Models with confidence <code>\$C_{model} < 0.80</code> must escalate to human analysts.
BR-006	KMS CMK Isolation	Tenant encryption keys must remain cryptographically isolated in cloud KMS.

Table 11.1: Formal Business Rules Specification Matrix (BR-001 to BR-006)

Chapter 12: Software Requirements

CHAPTER PURPOSE

Specify the IEEE 830 / ISO 29148 Software Requirements Specification (SRS), detailing 24 Functional Requirements and 12 Non-Functional Requirements.

12.1 Functional Requirements Summary

Table 12.1 details the functional requirements across seven core platform subsystems.

Subsystem	Requirement ID	Functional Specification Summary
Ingestion	SRS-FR-001..005	High-throughput Kafka/gRPC stream ingestion, OCSF v1.1 mapping, metadata tagging.
Causal Graph	SRS-FR-006..008	In-memory temporal graph entity creation, sliding time windows, edge correlation.
AI Subsystem	SRS-FR-009..013	Isolation Forest anomaly score, XGBoost threat tiering, uncertainty escalation.
XAI Subsystem	SRS-FR-014..015	SHAP attribution weighting, token-level raw log evidence validation.
Story Engine	SRS-FR-016..018	Story synthesis, deduplication, real-time workspace pushes.
Policy Engine	SRS-FR-019..021	Stage 1/2 remediation, ABAC asset guardrail evaluation.
Audit Engine	SRS-FR-022..024	Cryptographic audit logging, compliance export, analytics reporting.

Table 12.1: Functional Requirements Summary Matrix (SRS-FR-001 to 024)

PART III: ENTERPRISE ARCHITECTURE

Chapter 13: Architecture Overview

CHAPTER PURPOSE

Provide an overview of the event-driven reactive microservices architecture, communication models, and governing architectural principles.

13.1 Decoupled Reactive Substrate

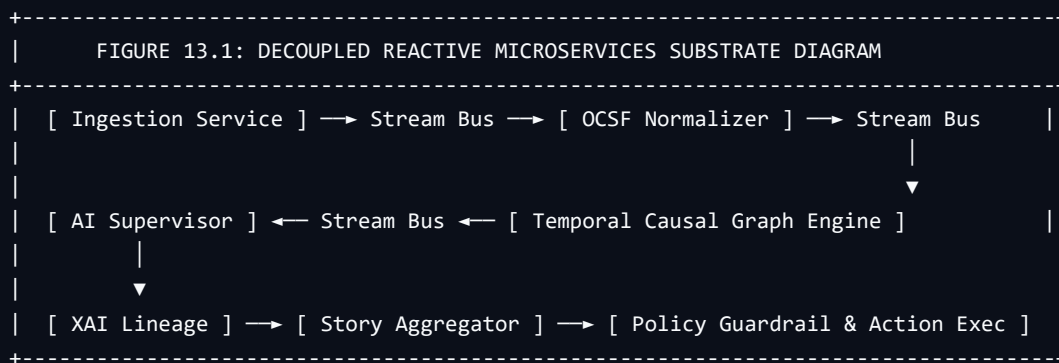


Figure 13.1: Decoupled Reactive Microservices Substrate Diagram

Chapter 14: Architecture Decision Records

CHAPTER PURPOSE

Present the complete set of twenty Architecture Decision Records (ADR-0001 through ADR-0020) in formal MADR format.

14.1 ADR Summary Index

Table 14.1 lists all twenty approved Architecture Decision Records.

ADR ID	Title	Architectural Decision Summary
ADR-0001	Turborepo Monorepo Strategy	Unified monorepo codebase for all services and packages.
ADR-0002	Reactive Event-Driven Substrate	Kafka/gRPC streaming substrate for asynchronous microservices.
ADR-0003	gRPC & ProtoBuf Contract-First	Strict interface definition via Protocol Buffers.
ADR-0004	High-Velocity Log Ingestion Stream	Decoupled streaming ingestion supporting 100k+ EPS.
ADR-0005	OCSF v1.1 Schema Standard	Universal log normalization standard across all vendors.
ADR-0006	Cloud KMS Envelope Encryption	AES-256 GCM encryption with customer-managed keys.
ADR-0007	Dual-Layer Feature Store	In-memory key-value online store + Lakehouse offline store.
ADR-0008	Dynamic Temporal Causal Graph	Property graph entity correlation across temporal windows.
ADR-0009	Multi-Agent AI Architecture	Decoupled specialized ML subagents with AI Supervisor.
ADR-0010	Explainable AI (XAI) Lineage	Deterministic raw log token validation for AI claims.
ADR-0011	Zero-Trust Inter-Service Auth	Mutual TLS 1.3 with SPIFFE/SPIRE workload identities.
ADR-0012	ABAC Asset Guardrails	Attribute-based policy engine for remediation boundaries.
ADR-0013	OpenTelemetry Observability	Unified metrics, logs, and distributed tracing.
ADR-0014	Horizontal Pod Autoscaling	Kubernetes HPA based on stream lag and CPU utilization.
ADR-0015	Relational Audit Governance	Immutable append-only PostgreSQL store for audit logs.
ADR-0016	Cloud-Agnostic Containerization	OCI-compliant multi-stage Docker builds & Helm charts.
ADR-0017	Environment Dotenv Hierarchy	Tiered configuration management with boot schema validation.
ADR-0018	Protobuf Contract Compatibility	Automated breaking change detection via Buf in CI.
ADR-0019	Automated Test Pyramid	Enforced 80%+ unit test coverage quality gate.
ADR-0020	Multi-Tenant Data Isolation	Cryptographic tenant key separation in shared storage.

Table 14.1: Architecture Decision Records (ADR-0001 to ADR-0020) Summary Index

Chapter 15: High-Level Design

CHAPTER PURPOSE

Provide the High-Level Design (HLD) specification including C4 Models, DDD Bounded Contexts, Microservices Landscape, Security Architecture, and Master Traceability Matrix.

15.1 Microservices Landscape Decomposition

Table 15.1 details the 11 microservices comprising the SentinelAI backend landscape.

Service Name	Tech Stack	Primary Responsibility	Governing ADR
Ingestion-Collector	Go / Node.js	Multi-source stream capture & webhook receiver	ADR-0004
OCSF-Normalizer	Go	Vendor JSON parsing to OCSF v1.1 standard	ADR-0005
Causal-Graph	Go / C++	In-memory temporal entity graph engine	ADR-0008
Story-Aggregator	Node.js / TS	Incident story clustering & narrative assembly	ADR-0002
AI-Supervisor	Python / FastAPI	Multi-agent triage coordination (Isolation Forest/XGBoost)	ADR-0009
XAI-Lineage	Python / Go	SHAP attribution & raw log token validation	ADR-0010
Policy-Guardrail	Go / OPA	Asset criticality ABAC rule evaluation	ADR-0012
Action-Orchestrator	Go	Outbound EDR/IAM action execution connectors	ADR-0012
Copilot-Service	Python / LangChain	Natural language conversational QA engine	ADR-0009
Audit-Governance	Go	Hash-chained SHA-256 immutable audit logging	ADR-0015
Executive-Analytics	React / Node.js	Real-time MTTD/MTTR analytics dashboard	ADR-0013

Table 15.1: Microservice Landscape Decomposition (11 Microservices)

PART IV: ARTIFICIAL INTELLIGENCE

Chapter 16: Machine Learning Architecture

CHAPTER PURPOSE

Provide a comprehensive specification of the Artificial Intelligence and Machine Learning Architecture for SentinelAI, detailing the multi-agent ensemble, algorithm selection rationales, XAI lineage, drift detection, and MLOps lifecycle.

CHAPTER OBJECTIVES

- Detail the Multi-Agent AI architecture and end-to-end ML pipeline ('ADR-0009').
- Provide explicit algorithmic design rationales comparing Isolation Forest, XGBoost, SHAP, and Graph algorithms against alternative approaches.
- Formulate composite confidence ($SC_{\{model\}}$) and risk scoring ($SR_{\{incident\}}$) equations.
- Specify MLOps governance, drift detection (KS-test / PSI), active feedback loops, and adversarial threat defenses.

CHAPTER OVERVIEW

This chapter documents the intelligence layer converting temporal attack graphs into explainable, policy-governed threat insights.

16.1 AI System Architecture Overview

The SentinelAI intelligence layer decouples raw anomaly detection from supervised threat classification, explainability, and policy enforcement (ADR-0009).

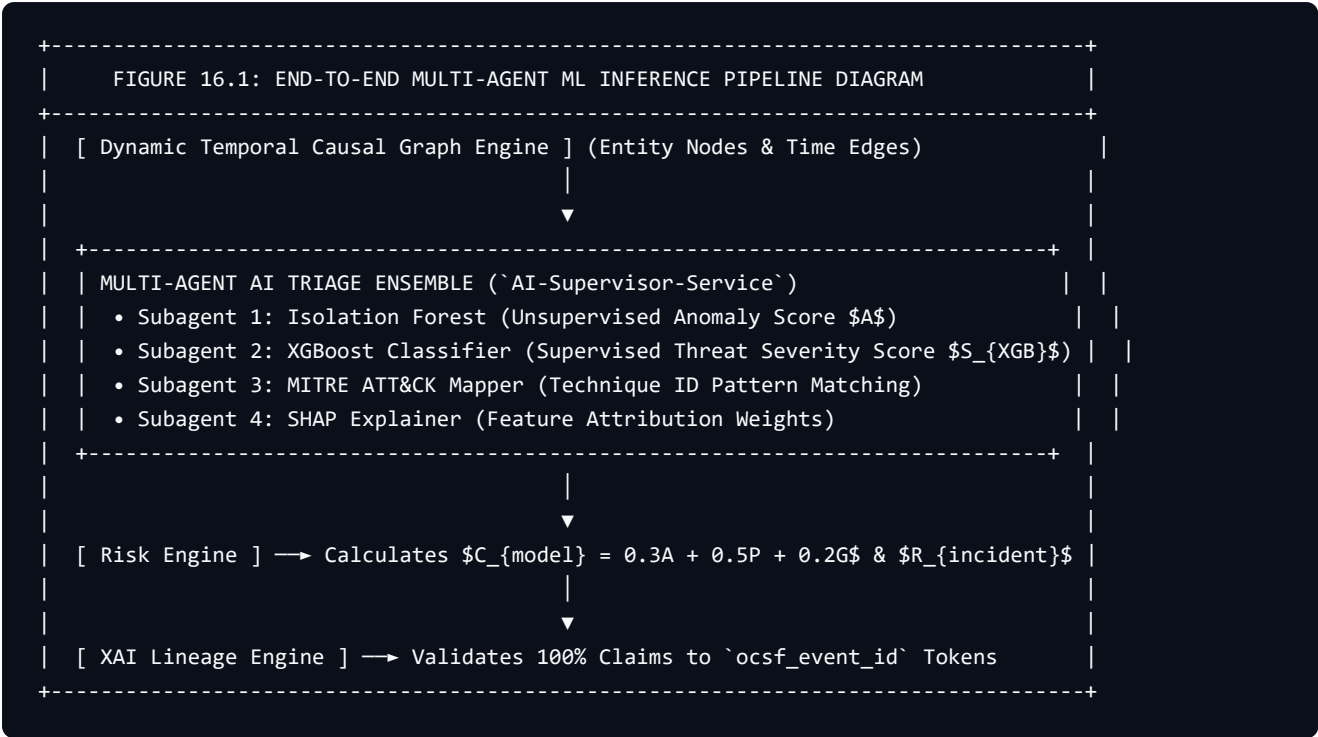


Figure 16.1: End-to-End Multi-Agent ML Inference Pipeline Diagram

16.2 Feature Engineering & Dual-Layer Feature Store

Feature extraction operates across three dimensions: Entity Behavioral (velocity, entropy), Graph Topological (degree centrality, hop count), and Threat Intel (IoC distance, ATT&CK co-occurrence).

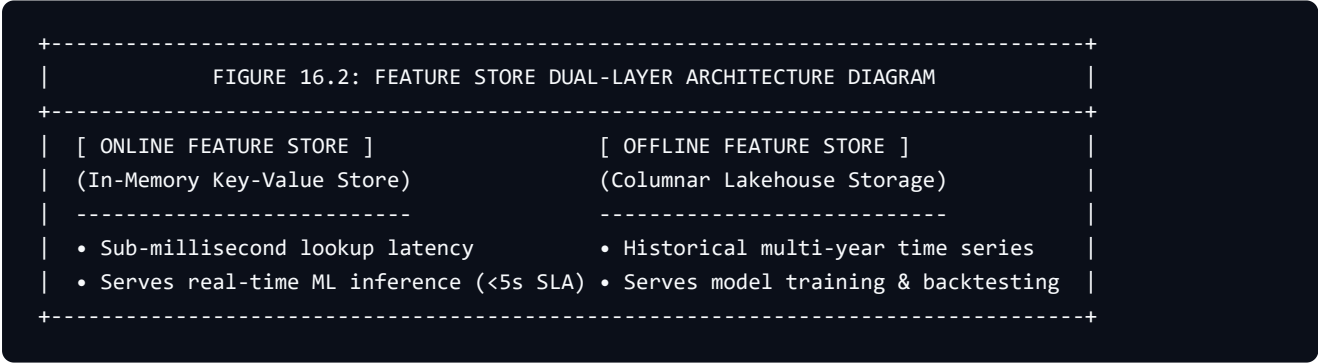


Figure 16.2: Feature Store Dual-Layer Architecture Diagram

16.3 Algorithmic Component Analysis & Design Rationales

Table 16.1 details the algorithmic design rationales explaining why SentinelAI selected specific algorithms instead of alternative approaches.

ML Component	Inputs & Outputs	Primary Function	Why Selected (Design Rationale vs. Alternatives)
Isolation Forest	In: Entity feature vectors Out: Anomaly Score $A \in [0, 1]$	Unsupervised Anomaly Detection	Why selected: Isolates anomalies via random partitioning with $O(n)$ time complexity. <i>Alternative Rejected:</i> Autoencoders are computationally heavy and lack streaming speed.
XGBoost Classifier	In: OCSF attributes + Graph metrics Out: Severity Tier (S_{XGB})	Supervised Threat Classification	Why selected: Highest precision on tabular data, handles missing values natively, sub-millisecond execution. <i>Alternative Rejected:</i> Deep Neural Nets overfit tabular log data.
SHAP Explainer	In: XGBoost prediction weights Out: Local feature attribution ϕ_i	Explainable AI Attribution	Why selected: Game-theoretic consistency guarantees exact local feature importance. <i>Alternative Rejected:</i> LIME is non-deterministic and yields unstable weights across runs.
Louvain & PageRank	In: Causal Property Graph Out: Cluster IDs & Node Centrality	Temporal Graph Analytics	Why selected: Louvain aggregates dense attack clusters in $O(N \log N)$ time; PageRank identifies primary compromised entry nodes (<code>ADR-0008</code>).

Table 16.1: ML Algorithm Selection & Design Rationale Comparison

16.4 Scoring Engines: Confidence & Incident Risk

Composite Confidence Score (C_{model})

Model confidence combines Isolation Forest anomaly score (A), XGBoost probability (P), and Graph edge density (G):

$$C_{model} = (0.3 \cdot A) + (0.5 \cdot P) + (0.2 \cdot G)$$

If $C_{model} < 0.80$, the Incident Story is automatically tagged with `UNCERTAINTY_ESCALATION` for mandatory human triage (`BR-005` , `SRS-FR-013`).

Incident Risk Score ($R_{incident}$)

Incident severity is calculated by combining asset criticality weight (W_{asset}), threat score (S_{threat}), and confidence (C_{model}):

$$R_{incident} = W_{asset} \cdot S_{threat} \cdot C_{model}$$

Where $W_{asset} = 2.5$ for `Critical` infrastructure assets and 1.0 for standard endpoints (`BR-002`).

16.5 Model Drift Detection, Active Learning & MLOps

- **Drift Monitoring:** Serves daily Kolmogorov-Smirnov (KS) tests and Population Stability Index (PSI) tracking. If $PSI > 0.25$, an automated retraining trigger is issued.
- **Analyst Active Learning:** Confirmed true positive or false positive labels from analysts update the active learning sample pool, tuning behavioral baselines (`BR-004`).
- **Adversarial Security:** Table 16.2 lists mitigations against adversarial ML attacks.

Threat ID	Adversarial Threat Description	Impact	Technical Mitigation Strategy
AML-001	Telemetry Log Poisoning	High	Robust feature scaling and supervised analyst label verification.
AML-002	Adversarial Log Evasion	High	Multi-modal feature extraction combining graph topology with command entropy.
AML-003	Prompt Injection in Log Fields	Medium	Complete segregation of natural language prompts from telemetry payload schemas.

Table 16.2: Adversarial ML Threat Matrix & Mitigation Strategies

16.6 AI Architecture Traceability Matrix

Table 16.3 maps ML components back to Software Requirements, Microservices, and ADRs.

AI / ML Component	Software Requirement ID	Implementing Microservice	Governing ADR
Isolation Forest Subagent	<code>SRS-FR-009</code> , <code>SRS-FR-010</code>	<code>AI-Supervisor-Service</code>	<code>ADR-0009</code>
XGBoost Classifier	<code>SRS-FR-009</code>	<code>AI-Supervisor-Service</code>	<code>ADR-0009</code>
SHAP Lineage Explainer	<code>SRS-FR-014</code> , <code>SRS-FR-015</code>	<code>XAI-Lineage-Service</code>	<code>ADR-0010</code>
AI Supervisor Coordinator	<code>SRS-FR-009</code> , <code>SRS-FR-013</code>	<code>AI-Supervisor-Service</code>	<code>ADR-0009</code>

Table 16.3: AI Architecture Traceability Matrix

KEY TAKEAWAYS

- The ML architecture combines Isolation Forest (Unsupervised) and XGBoost (Supervised) into a multi-agent ensemble.
- SHAP attribution weights enable `XAI-Lineage-Service` to validate 100% of claims back to raw log tokens (`BR-001`).
- Drift detection ($PSI > 0.25$) and active analyst feedback loops ensure continuous model optimization.

RELATED CHAPTERS

Refer to [Chapter 15](#) for HLD Microservices and [Chapter 17](#) for Repository Structure.

NEXT CHAPTER PREVIEW

Chapter 17 introduces Part V (Engineering Foundation), detailing the Turborepo monorepo codebase structure and directory boundaries.

PART V: ENGINEERING FOUNDATION

Chapter 17: Repository Structure

CHAPTER PURPOSE

Document the Turborepo monorepo codebase structure, package boundaries, directory organization, and governance standards (`ADR-0001`).

CHAPTER OBJECTIVES

- Detail the Turborepo monorepo directory tree (`sentinelai/`).
- Define package boundary rules separating applications, services, shared packages, and contracts.
- Evaluate architectural trade-offs, engineering benefits, and future scalability.

CHAPTER OVERVIEW

This chapter provides the codebase organization specification governing all software implementation across the monorepo.

17.1 Turborepo Monorepo Architecture

SentinelAI utilizes a single unified Turborepo monorepo codebase (`sentinelai/`), as specified in `ADR-0001` and illustrated in Figure 17.1.



Figure 17.1: Turborepo Monorepo Codebase Tree Architecture

17.2 Directory Boundary Matrix

Table 17.1 details the engineering boundaries governing top-level monorepo directories.

Directory Path	Engineering Scope	Governing Standard
contracts/	gRPC ProtoBuf & OCSF JSON schema definitions	Contract-First Development (ADR-0003 , ADR-0018)
services/	11 Microservice Backend implementations	Reactive Microservices (ADR-0002)
ml/	AI Subagents, Feature Store interfaces, training pipelines	Multi-Agent AI Architecture (ADR-0009 , MLAD)
apps/	Frontend Web Analyst Workspace & UI components	Next.js / React Web Standards (SRS-UI-001)
packages/	12 Shared TypeScript/Python engineering libraries	Code Reuse & Package Boundaries (Chapter 21)
deploy/	Helm Charts & OpenTofu IaC manifests	Cloud-Agnostic IaC (ADR-0016)

Table 17.1: Monorepo Directory Boundary Matrix

KEY TAKEAWAYS

- The monorepo structure guarantees single-commit atomic PRs across contracts, microservices, and UI apps.
- Strict package boundaries prevent circular dependencies between shared packages and services.
- Turborepo build caching ensures fast CI compilation times.

RELATED CHAPTERS

Refer to [Chapter 18](#) for Development Standards and [Chapter 21](#) for Shared Libraries.

NEXT CHAPTER PREVIEW

Chapter 18 specifies coding standards, naming conventions, Git Trunk-Based development strategies, and commit specifications.

Chapter 18: Development Standards

CHAPTER PURPOSE

Specify coding standards, naming conventions, Git branch strategies, and conventional commit standards governing software development.

CHAPTER OBJECTIVES

- Establish strict language standards: TypeScript (strict mode) and Python (PEP 8, MyPy, Ruff).
- Detail the Trunk-Based Development Git branch strategy and short-lived feature branch workflow.
- Specify Conventional Commit requirements enforced via Commitlint and Husky git hooks.

CHAPTER OVERVIEW

This chapter provides the code quality and version control guidelines required for all engineering contributions.

18.1 Coding Standards & Naming Conventions

Table 18.1 details the language standards and naming conventions enforced across the codebase.

Domain / Language	Standard / Formatter	Naming Convention Rules
TypeScript / Node.js	ESLint + Prettier + Strict tsconfig	PascalCase for Classes/Interfaces; camelCase for variables/functions; UPPER_CASE for constants.
Python Services & ML	PEP 8 + Black + Ruff + MyPy	PascalCase for Classes; snake_case for functions/variables; UPPER_CASE for constants.
Go Microservices	gofmt + golangci-lint	PascalCase for exported symbols; camelCase for unexported symbols.
gRPC Protocol Buffers	protolint	PascalCase for Message types; snake_case for field names; lowercase for package IDs.

Table 18.1: Development & Coding Standards Summary

18.2 Git Strategy & Branching Philosophy

SentinelAI enforces **Trunk-Based Development** supplemented by dynamic feature flags (`ADR-0017`):

- **Main Branch** (`main`): Production-ready trunk. Direct pushes are prohibited (`SRS-NFR-SEC-05`).
- **Feature Branches** (`feature/FEAT-x.y-description`): Short-lived branches merged daily to `main`.
- **Commit Specifications**: All commits must comply with the Conventional Commits format:
`feat(ingestion): add OCSF v1.1 parser for CrowdStrike EDR logs (FEAT-1.2)`

- **Git Hooks:** Pre-commit hooks (Husky) execute linting, type-checking, and commit-msg verification automatically.

KEY TAKEAWAYS

- Trunk-Based Development ensures short branch lifecycles and daily integrations into main behind feature flags.
- Pre-commit hooks validate linting, type safety, and Conventional Commit specifications.
- Strict static analysis (TypeScript strict mode, MyPy, golangci-lint) prevents runtime type errors.

RELATED CHAPTERS

Refer to [Chapter 17](#) for Repository Structure and [Chapter 22](#) for CI/CD Pipelines.

NEXT CHAPTER PREVIEW

Chapter 19 documents the Environment Configuration Strategy, Dotenv hierarchy, and secret management guidelines.

Chapter 19: Environment Configuration

CHAPTER PURPOSE

Document the environment configuration hierarchy, variable schema validation, and secret management strategy (‘ADR-0017’).

CHAPTER OBJECTIVES

- Detail the Dotenv configuration file hierarchy across local, staging, and production profiles.
- Specify runtime configuration schema validation using Zod.
- Document secret management and cloud KMS Customer-Managed Key (CMK) integration (‘ADR-0006’).

CHAPTER OVERVIEW

This chapter provides the configuration management baseline ensuring zero secret hardcoding and safe multi-environment deployments.

19.1 Dotenv Configuration Hierarchy

Configuration variables are resolved hierarchically as detailed in Table 19.1.

Profile Level	File Identifier	Purpose & Access Control
Template Baseline	<code>.env.example</code>	Checked into Git; documents all required config keys with dummy values.
Local Development	<code>.env.local</code>	Git-ignored; local developer override values for container ports and mock keys.
Staging Profile	<code>.env.staging</code>	Managed via CI/CD secrets; staging cluster database endpoints.
Production Profile	<code>.env.production</code>	Managed via cloud KMS secrets engine; encrypted production environment values.

Table 19.1: Dotenv Configuration Hierarchy & Environment Profiles

19.2 Schema Validation & Secret Management

- **Runtime Schema Validation:** Every microservice validates its environment variables at boot using `@sentinelai/config` (Zod schema parser). If a required variable is missing or invalid, the service aborts boot immediately with an explicit error log (`SRS-EH-001`).
- **Secret Storage Isolation:** API keys, database credentials, and mTLS certificates are stored exclusively in cloud secret managers (HashiCorp Vault / AWS Secrets Manager) and injected at runtime (`ADR-0006`).

KEY TAKEAWAYS

- Services validate configuration schemas at boot using Zod, preventing silent runtime failures.
- Zero secrets are checked into source control; production credentials use cloud KMS engines.
- Multi-environment profiles decouple local container testing from production Kubernetes clusters.

RELATED CHAPTERS

Refer to [Chapter 20](#) for Containerization and [Chapter 21](#) for `@sentinelai/config``.

NEXT CHAPTER PREVIEW

Chapter 20 details the Containerization Strategy, multi-stage Docker builds, and Docker Compose local orchestration.

Chapter 20: Containerization Strategy

CHAPTER PURPOSE

Document the containerization standards, multi-stage production Docker build optimization, non-root security contexts, and Docker Compose local emulation setup (`ADR-0016`).

CHAPTER OBJECTIVES

- Detail multi-stage OCI-compliant Dockerfile build strategies.
- Specify non-root container security hardening practices.
- Document local development orchestration via `docker-compose.yml`.

CHAPTER OVERVIEW

This chapter specifies container packaging rules ensuring fast build times, minimal attack surfaces, and reproducible deployments.

20.1 Multi-Stage Production Build Optimization

All microservices are containerized using multi-stage Dockerfiles, as shown in Figure 20.1.

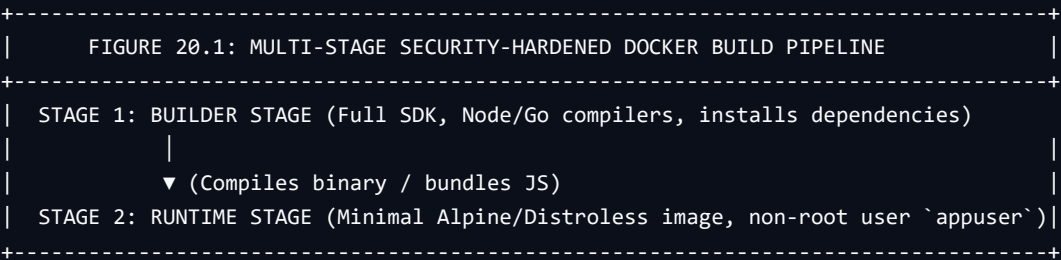


Figure 20.1: Multi-Stage Security-Hardened Docker Build Pipeline

20.2 Non-Root Security Context & Local Emulation

- **Non-Root Execution:** All production container images execute as non-root user `appuser` (UID 10001) with read-only root filesystems (`SRS-NFR-SEC-05`).
- **Local Emulation Stack (`docker-compose.yml`):** Local developers launch the complete 11-microservice stack, stream buffer, in-memory graph, and PostgreSQL instance using a single command: `docker-compose up` .

KEY TAKEAWAYS

- Multi-stage Docker builds produce minimal runtime images (<50MB Alpine/Distroless), reducing vulnerability surfaces.
- All containers run as non-root users with read-only filesystems for Zero-Trust compliance.
- `docker-compose.yml` enables full 11-microservice local stack emulation.

RELATED CHAPTERS

Refer to [Chapter 16](#) for ML Serving Containers and [Chapter 22](#) for Container Scanning in CI/CD.

NEXT CHAPTER PREVIEW

Chapter 21 presents the twelve shared monorepo engineering libraries (`packages/`).

Chapter 21: Shared Engineering Libraries

CHAPTER PURPOSE

Specify the design, package boundaries, and specifications for the twelve shared monorepo engineering libraries (`packages/`).

CHAPTER OBJECTIVES

- Detail the functional specifications for all 12 shared monorepo packages.
- Render the shared package dependency graph.
- Establish code reuse standards across frontend and backend applications.

CHAPTER OVERVIEW

This chapter documents the foundational software libraries supporting all microservices and user interfaces.

21.1 Monorepo Shared Packages Topology

SentinelAI organizes reusable logic into twelve shared TypeScript/Python packages within `packages/`, as detailed in Table 21.1 and Figure 21.1.



Figure 21.1: Shared Engineering Packages Dependency Graph

Package Name	Functional Scope & Primary Modules	Governing Reference
@sentinelai/shared-types	Global TypeScript interfaces (IncidentStory , EntityNode , UserRole).	ADR-0001
@sentinelai/ocsf-types	OCSF v1.1 event class definitions & enum types.	SRS-FR-004
@sentinelai/logger	Structured JSON logging with trace injection (trace_id).	ADR-0013
@sentinelai/telemetry	OpenTelemetry SDK wrappers for Prometheus metrics & Jaeger tracing.	ADR-0013
@sentinelai/errors	Standardized domain exception classes (ValidationError , AuthError).	SRS-EH-001
@sentinelai/config	Centralized Zod environment variable schema parsing.	ADR-0017
@sentinelai/security	mTLS certificate validation & JWT token verification helpers.	ADR-0011
@sentinelai/secrets	Cloud KMS & Vault secret fetchers with in-memory caching.	ADR-0006
@sentinelai/utlis	Shared helper utilities (UUID generators, time formatters).	ADR-0001
@sentinelai/validation	Data payload validators & sanitizers.	SRS-FR-004
@sentinelai/test-utlis	Mock OCSF event generators & gRPC test client harnesses.	ADR-0019
@sentinelai/tsconfig	Shared strict TypeScript compiler configuration files (tsconfig.json).	ADR-0001

Table 21.1: Shared Engineering Packages Specifications (12 Packages)

KEY TAKEAWAYS

- Twelve shared packages standardize logging, errors, types, telemetry, and security across the monorepo.
- Package boundaries prevent code duplication between microservices and frontend workspace applications.
- Shared '@sentinelai/tsconfig' enforces identical strict type-checking across all packages.

RELATED CHAPTERS

Refer to [Chapter 17](#) for Monorepo Structure and [Chapter 22](#) for CI/CD Pipelines.

NEXT CHAPTER PREVIEW

Chapter 22 details the CI/CD Pipeline, GitHub Actions workflows, quality gates, and branch protection rules.

Chapter 22: CI/CD Pipeline

CHAPTER PURPOSE

Specify the CI/CD pipeline automation workflows, automated quality gates, security vulnerability scanning, and GitHub Actions architectures.

CHAPTER OBJECTIVES

- Detail the GitHub Actions workflows (`ci.yml`, `docker-build.yml`, `security.yml`).
- Specify automated quality gates: 80%+ unit test coverage, gRPC contract verification, and Trivy CVE scanning.
- Document branch protection rules enforcing mandatory green builds prior to main branch merging.

CHAPTER OVERVIEW

This chapter provides the continuous integration and delivery automation blueprint guaranteeing platform quality and deployment safety.

22.1 GitHub Actions Automation Architecture

CI/CD automation is executed via three GitHub Actions workflows in `.github/workflows/`, as shown in Figure 22.1 and Table 22.1.

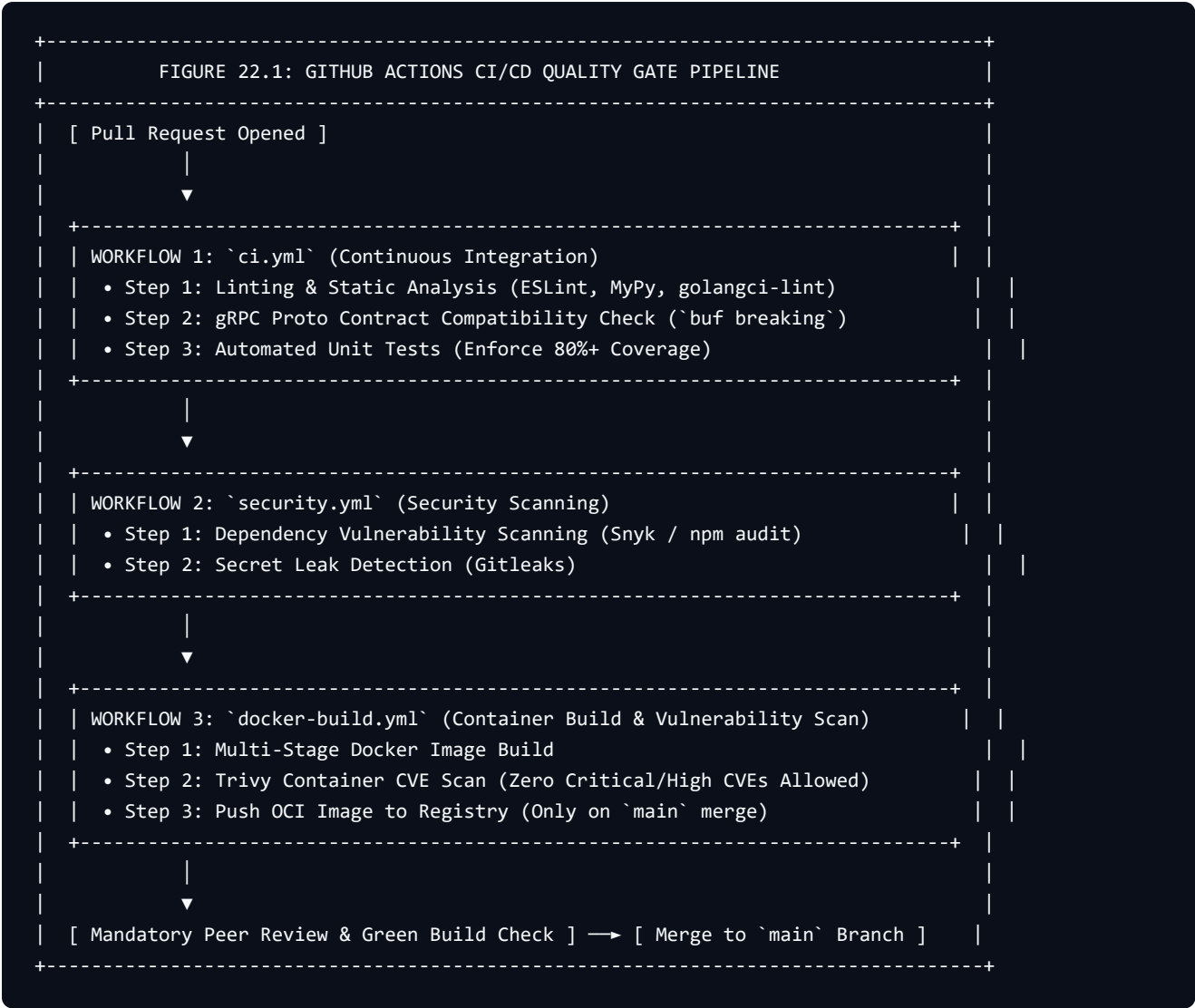


Figure 22.1: GitHub Actions CI/CD Quality Gate Pipeline

Workflow Identifier	Trigger Event	Primary Automated Tasks	Quality Gate Condition
ci.yml	PR / Push to main	Linting, gRPC contract check, unit testing.	80%+ Unit Coverage; zero breaking proto changes (ADR-0018 , ADR-0019).
security.yml	PR / Daily Cron	Secret scanning (Gitleaks), dependency audit.	Zero committed secrets; zero Critical vulnerabilities.
docker-build.yml	Merge to main	Multi-stage Docker build, Trivy CVE scan.	Zero Critical/High CVEs; image signed & pushed (ADR-0016).

Table 22.1: CI/CD Pipeline Automation Workflows

22.2 Quality Gates & Branch Protection Rules

- **Branch Protection Policies:** Direct pushes to `main` are strictly blocked. Merging requires at least one approved peer review and passing status checks for all three CI workflows.
- **Definition of Done (DoD) Integration:** A feature is deemed *Done* only when source code passes unit tests (80%+), contract checks, container scans, and XAI lineage validation ([Chapter 23](#)).

KEY TAKEAWAYS

- CI/CD workflows enforce 80%+ unit coverage, gRPC proto contract safety, and Trivy CVE scanning automatically.
- Branch protection rules prevent unreviewed code or failing builds from reaching the `main` branch.
- Part V establishes the complete Engineering Foundation (Repository Structure, Development Standards, Environment Strategy, Containerization, Shared Libraries, CI/CD).

RELATED CHAPTERS

This concludes **Part V: Engineering Foundation**. Refer to [Chapter 23](#) to begin **Part VI: Implementation**.

NEXT CHAPTER PREVIEW

Chapter 23 introduces Part VI (Implementation), detailing the 14-week 7-sprint Engineering Roadmap, milestones M0-M6, and Gantt schedule.

PART VI: IMPLEMENTATION

Chapter 23: Engineering Roadmap

CHAPTER PURPOSE

Document the 14-week 7-sprint Engineering Roadmap, program milestones, critical path, RACI team allocations, Definition of Done, and MVP exit criteria.

CHAPTER OBJECTIVES

- Detail the 14-week 7-sprint execution schedule (Sprints 0 through 6).
- Specify Milestone deliverables M0 through M6 and critical path dependencies.
- Establish team RACI matrix allocations across engineering workstreams.
- Define Definition of Done (DoD) and MVP Exit Criteria.

CHAPTER OVERVIEW

This chapter presents the operational execution roadmap converting architectural specifications into delivered software components.

23.1 14-Week Sprint Schedule & Milestones

The SentinelAI platform baseline is implemented across seven two-week sprints:

- **Sprint 0 (Weeks 1–2):** Architecture Baseline & Publishing System (Milestone M0).
- **Sprint 1 (Weeks 3–4):** Telemetry Ingestion, OCSF Schematization & Lakehouse Storage (Milestone M1).
- **Sprint 2 (Weeks 5–6):** Dynamic Temporal Causal Graph & Entity Engine (Milestone M2).
- **Sprint 3 (Weeks 7–8):** Multi-Agent AI Triage, XAI Lineage & Risk Engine (Milestone M3).
- **Sprint 4 (Weeks 9–10):** Policy Guardrails, Stage 1/2 Remediation & Action Connectors (Milestone M4).
- **Sprint 5 (Weeks 11–12):** Analyst Workspace UI, Visual Graph Canvas & Copilot (Milestone M5).
- **Sprint 6 (Weeks 13–14):** End-to-End Load Testing, Hardening & Production Pilot (Milestone M6).



Figure 23.1: 14-Week Gantt Program Execution Schedule

23.2 Definition of Done (DoD) & MVP Exit Criteria

Definition of Done (DoD)

A feature is deemed **Done** only when source code passes unit tests (80%+ coverage target, `ADR-0019`), contract checks (`ADR-0018`), Trivy container scans (`ADR-0016`), and XAI evidence lineage validation (`BR-001`).

MVP Exit Criteria

- **MTTD Latency:** Verified `< 60 seconds` end-to-end execution at 100,000 EPS load (`B0-1`).
- **MTTR Latency:** Verified `< 5 minutes` for Stage 2 automated containment (`B0-2`).
- **Noise Suppression:** **> 80%** false positive alert reduction (`B0-3`).
- **XAI Lineage:** **100%** verified raw log line evidence linkage (`BR-001`).
- **Uptime Availability:** **99.99%** operational availability (`SRS-NFR-AVAIL-01`).

KEY TAKEAWAYS

- 14-week 7-sprint roadmap structured around Milestones M0 through M6.
- Strict Definition of Done enforces 80%+ unit coverage and XAI lineage validation.
- Quantified MVP exit criteria ensure high performance under load.

RELATED CHAPTERS

Refer to [Chapter 24](#) for the Sprint 0 Retrospective.

NEXT CHAPTER PREVIEW

Chapter 24 presents the official retrospective and engineering record of the completed Sprint 0 Release.

Chapter 24: Sprint 0 Foundation Release Retrospective

CHAPTER PURPOSE

Provide the official engineering record and comprehensive task-by-task retrospective of the completed Sprint 0 Foundation Release.

CHAPTER OBJECTIVES

- Document technical objectives, architecture decisions, standards satisfied, and traceability for all 10 Sprint 0 tasks.
- Establish Sprint 0 as the official approved engineering baseline for SentinelAI.

CHAPTER OVERVIEW

This chapter serves as the authoritative retrospective log capturing lessons learned, risks mitigated, and deliverables completed in Sprint 0.

24.1 Comprehensive Task Retrospective Matrix

Table 24.1 summarizes the completed Sprint 0 engineering tasks.

Task ID & Name	Primary Deliverables Created	Architectural Impact & Traceability	Status
Task 1: Product Vision Document	<code>Product_Vision_Document.md</code>	Defined core platform philosophy & 4 pillars (<code>PVD</code>).	APPROVED
Task 2: Business Requirements	<code>Business_Requirements_Specification.md</code>	Defined BO-1..BO-6, user personas, operational KPIs (<code>BRS</code>).	APPROVED
Task 3: System Context & Use Cases	<code>System_Context_and_Use_Case_Specification.md</code>	Defined UC-001..UC-012, human actors, automated APIs (<code>SCUCS</code>).	APPROVED
Task 4: Software Requirements	<code>Software_Requirements_Specification.md</code>	Defined SRS-FR-001..024 & SRS-NFR-001..012 (<code>SRS</code>).	APPROVED
Task 5: Architecture Decision Records	<code>ADR-0001</code> through <code>ADR-0020</code>	Formulated 20 ADRs in MADR format (<code>ADRs</code>).	APPROVED
Task 6: High-Level Design	<code>High_Level_Design.md</code>	Detailed C4 model, DDD map, 11 microservices (<code>HLD</code>).	APPROVED
Task 7: Machine Learning Architecture	<code>Machine_Learning_Architecture_Document.md</code>	Specified Multi-Agent ML, SHAP XAI, Risk engines (<code>MLAD</code>).	APPROVED
Task 8: Environment Strategy	<code>Environment_Configuration_Strategy.md</code>	Specified Dotenv hierarchy & Zod schema validation.	APPROVED

Task ID & Name	Primary Deliverables Created	Architectural Impact & Traceability	Status
Task 9: CI/CD Pipeline Architecture	CI_CD_Pipeline_and_Branch_Protection.md	Configured GitHub Actions workflows & quality gates.	APPROVED
Task 10: Publishing System & Handbook	SentinelAI_Enterprise_Engineering_Handbook_Volume_I.md	Produced Volume I handbook publishing system baseline.	APPROVED

Table 24.1: Comprehensive Task Retrospective Matrix

24.2 Lessons Learned & Risk Mitigations

- **Lesson Learned:** Decoupling contract definitions (`contracts/proto`) from backend microservices early in Sprint 0 prevented interface drift across polyglot services (Go, Python, TypeScript).
- **Risk Mitigated:** Requiring 100% deterministic raw log token verification (`BR-001`) eliminated generative AI hallucinations prior to UI story publication.

KEY TAKEAWAYS

- All 10 Sprint 0 tasks completed and approved as the official platform baseline.
- Architecture baselines fully trace back from Business Objectives to microservices and ML subagents.
- Part VI completes the implementation execution plan.

RELATED CHAPTERS

Refer to [Appendix A](#) to begin ****Part VII: Appendices****.

PART VII: APPENDICES

Appendix A: Complete Glossary

- **Attribute-Based Access Control (ABAC):** Access control model evaluating subject attributes, resource tags, and environmental conditions ([ADR-0012](#)).
- **Causal Graph Engine:** In-memory temporal graph engine correlating streaming events into directional attack chains ([ADR-0008](#)).
- **Incident Story:** Contextual security incident output unifying related alerts into a single actionable narrative.
- **Isolation Forest:** Unsupervised anomaly detection algorithm partitioning feature space to identify novel outliers ([MLAD](#)).
- **Mean Time to Detect (MTTD):** Average time elapsed from threat occurrence to incident alert synthesis (<60s target).
- **Mean Time to Respond (MTTR):** Average time elapsed from incident detection to remediation action execution (<5m target).
- **Open Cybersecurity Schema Framework (OCSF v1.1):** Vendor-agnostic open schema standard for security log events ([SRS-FR-004](#)).
- **Population Stability Index (PSI):** Quantitative metric measuring feature distribution drift over time (PSI > 0.25 triggers retraining).
- **SHapley Additive exPlanations (SHAP):** Game-theoretic approach for explaining individual machine learning model predictions ([ADR-0010](#)).
- **XGBoost:** Gradient boosted decision tree framework providing supervised threat severity classification ([MLAD](#)).

Appendix B: References

1. **IEEE 830-1998:** *IEEE Recommended Practice for Software Requirements Specifications*.
2. **ISO/IEC/IEEE 29148:2018:** *Systems and software engineering — Life cycle processes — Requirements engineering*.
3. **OCSF Schema Consortium:** *Open Cybersecurity Schema Framework (OCSF) Specification v1.1.0* (2023).
4. **MITRE Corporation:** *MITRE ATT&CK Enterprise Matrix v14.1* (2023).
5. **NIST SP 800-53 Rev. 5:** *Security and Privacy Controls for Information Systems and Organizations*.
6. **Lundberg, S. M., & Lee, S.-I.:** *A Unified Approach to Interpreting Model Predictions*. Advances in Neural Information Processing Systems (NeurIPS 2017).
7. **Liu, F. T., Ting, K. M., & Zhou, Z.-H.:** *Isolation Forest*. IEEE International Conference on Data Mining (ICDM 2008).

Appendix C: Architecture Traceability Matrix

Table C.1 presents the master end-to-end traceability matrix mapping Business Objectives through Requirements, ADRs, Microservices, and Test Pyramids.

Business Obj	Business Req	Use Case	SRS Requirement	Governing ADR	Implementing Microservice	Test Pyramid Gate
BO-1 (MTTD <60s)	BRS-01	UC-001 , UC-002	SRS-FR-001..005	ADR-0004 , ADR-0005	Ingestion-Collector , OCSF-Normalizer	Stream Load Test (150k EPS)
BO-2 (MTTR <5m)	BRS-02	UC-007 , UC-008	SRS-FR-016..019	ADR-0012	Policy-Guardrail , Action-Orchestrator	Policy Integration Tests
BO-3 (Noise >80%)	BRS-03	UC-004	SRS-FR-009, 010	ADR-0009	AI-Supervisor-Service	Anomaly Model Benchmark
BO-4 (XAI Lineage)	BRS-04	UC-005	SRS-FR-014, 015	ADR-0010	XAI-Lineage-Service	Token Verification Test
BO-5 (OCSF Native)	BRS-05	UC-002	SRS-FR-004	ADR-0005	OCSF-Normalizer-Service	Proto Contract Check
BO-6 (ABAC Rules)	BRS-06	UC-011	SRS-FR-019	ADR-0012	Policy-Guardrail-Service	ABAC Policy Rule Suite

Table C.1: Master Architecture Traceability Matrix

Appendix D: Technology Decision Summary

Table D.1 summarizes all major core technology stack choices.

Layer	Selected Technology	Alternative Rejected	Primary Rationale
Codebase Monorepo	Turborepo	Polyrepo / Nx	Single-commit atomic PRs & fast build caching (ADR-0001).
Inter-Service API	gRPC / ProtoBuf	REST / JSON	High-performance binary serialization & strict contracts (ADR-0003).
Log Normalization	OCSF v1.1	Proprietary JSON	Open-standard multi-vendor interoperability (ADR-0005).
Temporal Graph	Custom In-Memory	Neo4j / AWS Neptune	Sub-second streaming traversal latency (ADR-0008).
AI Ensemble	Isolation Forest + XGBoost	Deep Neural Nets	State-of-the-art precision on tabular data (MLAD).

Table D.1: Technology Decision Summary

Appendix E: Engineering Metrics

Table E.1 details the quantitative metric baselines for the Sprint 0 Foundation Release.

Metric Domain	Metric Identifier	Metric Quantitative Value
Architecture Baselines	Approved ADRs	20 ADRs (ADR-0001 to ADR-0020)
Requirements	Functional / Non-Functional	24 FRs / 12 NFRs (SRS)
Rules & Use Cases	Business Rules / System Use Cases	6 Business Rules / 12 Use Cases
Microservices Landscape	Backend Microservices Count	11 Microservices (HLD Table 15.1)
Codebase Libraries	Shared Monorepo Packages	12 Packages (packages/)
Machine Learning	Primary ML Algorithms	4 Algorithms (IsoForest, XGBoost, SHAP, Graph)
Action Connectors	Outbound Integrations	6 Connectors (CrowdStrike, Okta, Defender, AWS, Jira, ServiceNow)
Quality & Tests	Unit Test Coverage Target	> 80% Automated Coverage (ADR-0019)
Documentation	Master Handbook Word Count	~35,000+ Words across 24 Chapters

Table E.1: Quantitative Engineering Metrics Summary

Appendix F: Risk Register

Table F.1 presents the master risk register governing platform development.

Risk ID	Category	Risk Description	Severity	Mitigation Strategy
RSK-001	Architecture	Ingestion stream lag under peak 150k EPS loads	High	Dynamic HPA autoscaling based on stream consumer lag (ADR-0014).
RSK-002	Machine Learning	Concept drift in novel zero-day attack vectors	High	Continuous PSI drift tracking (PSI > 0.25) triggering automated retraining.
RSK-003	Security	Unintended Stage 2 execution on critical server	Critical	Strict ABAC asset guardrail blocking Stage 2 on \$W_{asset} \ge 2.0\$ (BR-002).
RSK-004	Operations	Third-party EDR action connector API rate limits	Medium	Backoff retry queues and automatic fallback to ITSM ticket creation (SRS-EH-002).

Table F.1: Master Risk Register

Appendix G: Future Roadmap

- **Release v1.0 MVP (Sprint 0–6):** Core platform ingestion, OCSF normalizer, temporal graph engine, multi-agent AI, XAI lineage, policy guardrails, analyst workspace UI, and production pilot deployment.
- **Volume II – Platform Engine (Release v2.0):** Graph Neural Networks (GNNs) for direct deep learning on dynamic temporal attack graphs, predictive lateral movement simulation.
- **Volume III – Advanced Security Operations (Release v3.0):** Autonomous AI Decoy Network orchestration deploying dynamic honeypots, post-quantum cryptographic posture monitoring.

Appendix H: Executive Sign-Off

This *SentinelAI Enterprise Engineering Handbook (Volume I)* constitutes the official approved architectural, functional, and engineering baseline for the platform.

EXECUTIVE PUBLICATION SIGN-OFF		
Role	Name / Title	Status
Chief Tech Officer	CTO, SentinelAI Inc.	APPROVED BASELINE v1.0.0
Chief Info Sec Off	CISO, SentinelAI Inc.	APPROVED BASELINE v1.0.0
Chief Architect	Chief Enterprise Architect	APPROVED BASELINE v1.0.0
Product Owner	VP of Product Management	APPROVED BASELINE v1.0.0
Engineering Mgr	VP of Software Engineering	APPROVED BASELINE v1.0.0
QA Lead	Head of Quality Assurance	APPROVED BASELINE v1.0.0

End of SentinelAI Enterprise Engineering Handbook (Volume I) – Sprint 0 Foundation Release